WRITTEN TEST

ASP.NET Core & MySQL Software Engineer

Duration: 2 hours 30 minutes	Total Marks: 100

Instructions

- Answer all questions. Clearly state any assumptions.
- Write production-oriented solutions and explain important design decisions.
- For SQL questions, assume MySQL 8.0 or later unless otherwise stated.
- For ASP.NET Core questions, assume a modern ASP.NET Core Web API using Entity Framework Core.

SECTION A — MYSQL & DATABASE ENGINEERING

35 Marks

Q1. Top 3 Highest-Paid Employees per Department (6 marks)

Table: Employee(Id, Name, DepartmentId, Salary, JoiningDate)

Write one SQL query that returns the three highest-paid employees in each department.

Requirements:

- Do not use LIMIT to select the top employees per department.
- Use a window function where appropriate.
- Explain how ties in salary are handled and how you would return exactly three rows versus including ties.

Q2. Seven Consecutive Login Days (6 marks)

Table: UserLogin(UserId, LoginDate)

Write a MySQL 8.0 query that identifies users who logged in on seven consecutive calendar days. Return UserId, StartDate, and EndDate.

Consider duplicate login records for the same user on the same date. State whether overlapping seven-day periods should be returned.

Q3. Customer Order Analytics with Window Functions (6 marks)

Table: Orders(Id, CustomerId, OrderDate, Amount)

For every order, return:

- Current order amount
- Previous order amount for the same customer
- Difference from the previous order
- Running total amount for the customer

Use a single SQL statement and define a deterministic ordering for orders placed at the same time.

Q4. Index-Friendly Query Optimization (8 marks)

The following query is slow:

```
SELECT *
FROM Orders o
JOIN Customers c ON c.Id = o.CustomerId
JOIN Products p ON p.Id = o.ProductId
WHERE YEAR(o.OrderDate) = 2026
AND MONTH(o.OrderDate) = 6
```

```
AND c.Country = 'BD'
ORDER BY o.OrderDate DESC;
```

Rewrite it to make the date filter index-friendly. Recommend appropriate indexes, explain column order in composite indexes, and describe how you would use EXPLAIN ANALYZE to validate the changes.

Q5. Secure Dynamic Search and Pagination (9 marks)

Design a stored procedure or parameterized query for optional filters on CustomerName, DateFrom, DateTo, MinAmount, MaxAmount, Status, Country, and City. It must support SortBy, SortDirection, Page, and PageSize.

Requirements:

- Prevent SQL injection, including through sorting parameters.
- Use parameterized values and allowlist sort columns/directions.
- Support server-side pagination and a separate total count.
- Keep predicates index-friendly where possible.
- Explain how you would test performance for common filter combinations.

SECTION B — ASP.NET CORE & BACKEND ENGINEERING

40 Marks

Q6. Preventing Duplicate Subscriptions under Concurrency (8 marks)

Review this simplified logic:

```
var user = await db.Users.FirstOrDefaultAsync(x => x.Mobile == request.Mobile);
if (user == null) return BadRequest();
if (user.IsSubscribed) return BadRequest();
user.IsSubscribed = true;
await db.SaveChangesAsync();
return Ok();
```

Rewrite or redesign it to prevent duplicate subscriptions when 1,000 concurrent requests target the same user. Discuss race conditions, database constraints/transactions, optimistic or pessimistic concurrency, idempotent behavior, and suitable HTTP responses.

Q7. Payment API Design (10 marks)

Design POST /api/payments for a payment provider integration. Your solution must address:

- Idempotency-Key handling and persistence
- Duplicate payment prevention
- Transaction boundaries and payment state transitions
- Safe retries, timeouts, and uncertain provider responses
- Distributed locking: when it is useful and why it is not a replacement for database constraints
- Webhook verification and duplicate webhook delivery
- Appropriate HTTP status codes and error responses

Provide the main endpoint flow, data model outline, and pseudocode or C# implementation for the critical path.

Q8. Generic Repository / Query Service (7 marks)

Implement or outline a reusable data-access abstraction that supports dynamic filtering, includes, sorting, pagination, and projection. Explain how you would keep it type-safe, avoid exposing arbitrary client-supplied expressions, prevent over-fetching, and handle cancellation tokens and asynchronous execution.

Q9. Dynamic LINQ Sorting (5 marks)

Implement an IQueryable<T> extension that supports calls such as:

query.OrderBy("Name")

query.OrderBy("Salary desc")

query.OrderBy("JoiningDate")

Use expression trees or another safe approach. Validate property names, support ascending/descending order, and explain how invalid input should be handled.

Q10. Request/Response Logging Middleware (5 marks)

Implement ASP.NET Core middleware that records a correlation ID, execution time, and unhandled exceptions. Explain how request and response bodies can be captured without breaking the pipeline.

Your design must address stream buffering/restoration, large payloads, cancellation, and sensitive-data redaction. Do not log passwords, access tokens, payment credentials, or unnecessary personal data.

Q11. API Reliability and Observability (5 marks)

A production API occasionally returns 500 errors and experiences slow responses under load. Describe a practical investigation plan covering structured logs, correlation IDs, metrics, distributed tracing, database query analysis, dependency timeouts, connection pooling, and load testing. Identify which metrics and alerts you would introduce first.

SECTION C — C# & LINQ

15 Marks

Q12. Customer Order Summary (7 marks)

Given List<Order>, write a single LINQ query that returns one record per customer containing CustomerId, TotalOrders, HighestOrderAmount, AverageOrderAmount, and LatestOrderDate. State how customers with no orders should be handled if customers and orders are separate collections.

Q13. Department Salary Statistics (8 marks)

Given List<Employee>, return each department's highest salary, lowest salary, average salary, and median salary using LINQ/C#.

Explain how your implementation handles an even number of employees and an empty department. Consider time complexity and avoid repeatedly sorting the same data unnecessarily.

SECTION D — API DESIGN & PERFORMANCE

10 Marks

Q14. Product Search API (5 marks)

Design GET /api/products with keyword search, price range, multiple categories, multiple brands, sorting, pagination, and total record count.

Provide the Entity Framework Core query outline. Explain how you would avoid SQL injection, validate page size and sort fields, use AsNoTracking/projection, and ensure count and page queries are efficient.

Q15. High-Volume API Scenario (5 marks)

A product catalogue contains 1 million records and receives heavy search traffic. Describe how you would implement efficient server-side pagination and a responsive client experience. Discuss offset versus keyset pagination, debounced search, cancellation of obsolete requests, caching, suitable indexes, and why the API should not return all records at once.